

AGM

Sistema de Gestión Académica

Arquitectura de Microservicios — Proyecto Final · Servicios Web

Aarón Ulises Torres Corte

Alfredo Escudero Rivera

Alejandro Amador Lagunes

Héctor Alejandro Mendoza Merino

Johan Yuri Martínez García

Joselyn Ramírez Lima

Contexto y Objetivos del Proyecto

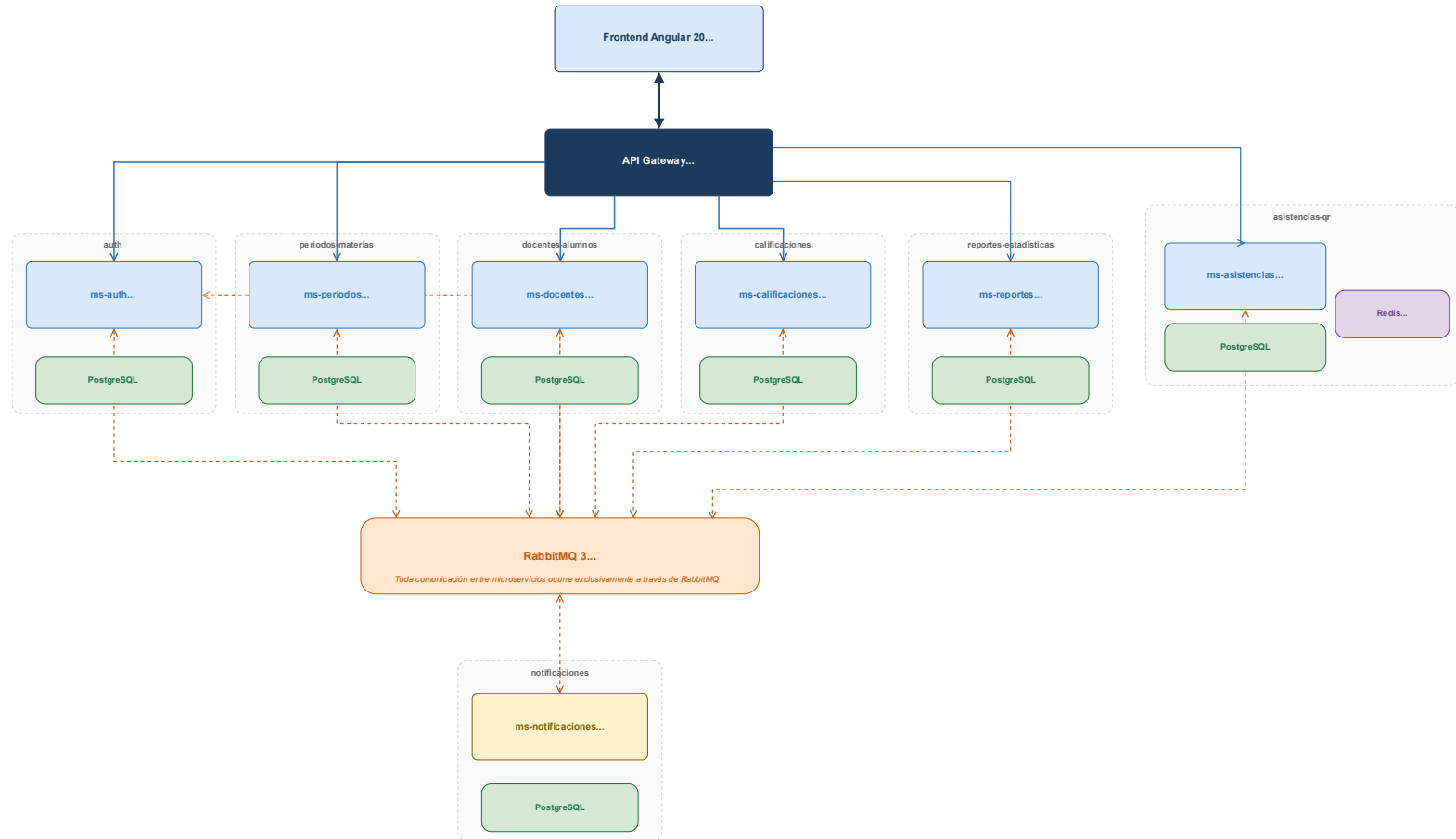
PROBLEMA

- 1 La gestión académica de periodos, materias, docentes y alumnos se realiza de forma fragmentada, sin integración entre módulos.
- 2 Los reportes de calificaciones y asistencias se generan manualmente, con alta probabilidad de error y sin trazabilidad.
- 3 No existe un mecanismo automatizado para notificar a docentes y alumnos sobre eventos relevantes del sistema.
- 4 El registro de asistencias es manual o por hoja de papel, sin control de tiempo ni validación de identidad.

SOLUCION: AGM

- A 7 microservicios independientes, cada uno con su propia base de datos, Dockerfile y puerto, coordinados por un API Gateway.
- B Comunicación interna vía RabbitMQ: RPC síncrono para operaciones que requieren respuesta inmediata y Pub/Sub asíncrono para eventos.
- C Generación automática de reportes de calificaciones y asistencias en formatos PDF y Excel.
- D Control de asistencias mediante código QR con ventana de tiempo, anti-replay y registro en tiempo real.

Diagrama de Arquitectura General del Sistema



Estrategia de Comunicación entre Microservicios

RPC Síncrono

RPC

Mecanismo: RabbitMQ como transporte RPC
Patrón request-reply con correlation ID y timeout de 10 segundos.
El servicio solicitante espera la respuesta antes de continuar.

CUANDO SE UTILIZA

- Validar token JWT desde auth_middleware de cualquier MS
- Obtener datos de una materia por ID o NRC
- Consultar alumnos inscritos en una materia
- Recuperar estadísticas de asistencia para reportes
- Verificar inscripción de alumno antes de registrar asistencia

Pub/Sub Asíncrono

EVT

Mecanismo: RabbitMQ topic exchange 'events_exchange'
Fire-and-forget: el MS publicador no espera respuesta.
ms-notificaciones consume todos los eventos y envía correos.

CUANDO SE UTILIZA

- Bienvenida a un alumno recién inscrito
- Notificación de baja de materia al docente
- Aviso de cierre de materia a los alumnos
- Enlace de recuperación de contraseña
- Notificación de retardo en asistencia
- Aviso de reporte listo para descarga

Flujos Asíncronos — Bus de Eventos (events_exchange)

DISPARADOR	ROUTING KEY	PUBLICADO POR	DESTINATARIO	ACCION
Inscripción de alumno	<code>periodos.bienvenida</code>	ms-periodos-materias	ms-notificaciones	Correo de bienvenida al alumno
Baja de materia	<code>docentes.baja</code>	ms-docentes	ms-notificaciones	Notificación al docente responsable
Cierre de materia	<code>periodos.materia_cerrada</code>	ms-periodos-materias	ms-notificaciones	Aviso de cierre a alumnos inscritos
Reset de contraseña	<code>auth.reset_password</code>	ms-auth	ms-notificaciones	Enlace de recuperación de contraseña
Retardo en asistencia	<code>asistencias.retardo</code>	ms-asistencias	ms-notificaciones	Notificación de retardo al docente
Reporte generado	<code>reportes.generado</code>	ms-reportes	ms-notificaciones	Aviso de reporte listo para descarga

Principio clave: ms-notificaciones consume todos los eventos pero nunca es llamado directamente por ningún otro MS. Si falla, el sistema completo sigue operando — solo se pierden las notificaciones por correo.

Independencia y Desacoplamiento entre Microservicios

1 BD Propia

Cada microservicio tiene su propia base de datos PostgreSQL. Ningún MS accede directamente a la BD de otro. La comunicación de datos se hace exclusivamente por RabbitMQ RPC.

2 Dockerfile Propio

Cada MS se construye y despliega como un contenedor independiente. Puede actualizarse, reiniciarse o escalar sin afectar a los demás.

3 Puerto Propio

Cada MS escucha en un puerto dedicado (8000, 8001, 8003, 8004, 8005, 8007). El gateway en Railway enruta las peticiones REST al MS correcto según el prefijo de ruta URL.

4 Bus Asíncrono

Los eventos Pub/Sub garantizan que un MS no crítico (como ms-notificaciones) pueda caer sin detener el flujo principal del sistema.

PRUEBA DE FUEGO

Si ms-notificaciones se cae durante el proceso de inscripción de un alumno o generación de un reporte, la operación principal se completa con éxito. El evento queda perdido, pero el sistema no falla en cascada.

Infraestructura del Sistema — Despliegue en Railway

PostgreSQL 15

PERSISTENCIA DE DATOS

Siete instancias de base de datos, una por microservicio. Garantiza el aislamiento de datos entre servicios. Puerto: 5432.

RabbitMQ 3

BUS DE MENSAJERÍA

Motor central de comunicación. AMQP en :5672. Consola de administración en :15672. Soporta RPC y Pub/Sub.

Redis 7

CACHÉ EN MEMORIA

Almacena sesiones de asistencia activas y tokens QR con TTL. Puerto: 6379. Alta velocidad de lectura/escritura.

SMTP4dev

SERVIDOR SMTP DE PRUEBAS

Captura correos enviados por ms-notificaciones en el entorno de desarrollo. Interfaz web en :5000.

Railway Gateway

API GATEWAY EN PRODUCCION

Punto de entrada único para el frontend. Enruta peticiones REST al MS correspondiente por ruta URL. Puerto: 80.

Docker Compose

ORQUESTACIÓN DE CONTENEDORES

Define y coordina los 7 MS + 5 servicios de infraestructura. Configura redes internas, volúmenes y dependencias de arranque.

Stack Tecnológico del Proyecto

BACKEND

- FastAPI (Python 3.12) — 5 microservicios
- Django 5 (Python 3.12) — ms-periodos-materias
- Node.js 20 + Express — ms-notificaciones
- SQLAlchemy ORM · Django ORM

FRONTEND

- Angular 20 (TypeScript)
- Angular Material M3
- Reactive Forms + Guards
- Desplegado en Vercel

MENSAJERÍA

- RabbitMQ 3 (protocolo AMQP)
- RPC síncrono con correlation ID
- Topic exchange para Pub/Sub
- Timeout configurable por llamada

PERSISTENCIA

- PostgreSQL 15 (×7 bases de datos)
- Redis 7 (sesiones y tokens QR)
- Volúmenes Docker para persistencia
- Migraciones por ORM en cada MS

SEGURIDAD

- JWT HS256 con roles en payload
- Roles: Administrador · Docente · Alumno
- Protección anti-IDOR en calificaciones
- Anti-replay en tokens QR de asistencias

DEVOPS

- Docker + Docker Compose (desarrollo local)
- Railway: despliegue del backend en la nube
- GitHub (rama dev, flujo de trabajo en equipo)
- Vercel: despliegue del frontend (Angular 20)

FRAMEWORK

FastAPI · Python 3.12

PUERTO

:8000

BASE DE DATOS

agm_auth_db

DESCRIPCION DEL MICROSERVICIO

Microservicio responsable de la autenticación y autorización de todos los usuarios del sistema. Recibe las credenciales del usuario, las valida contra su base de datos y, si son correctas, genera un token JWT firmado con HS256 que incluye el identificador del usuario, su rol y la fecha de expiración.

El token generado es utilizado por todos los demás microservicios para identificar y autorizar al usuario en cada petición. Cada MS tiene un `auth_middleware` que llama a `rpc_auth_queue` para validar el token sin necesidad de tener acceso directo a la BD de autenticación.

Roles soportados: Administrador, Docente, Alumno.

MÉTODO	RUTA	ROL REQUERIDO	DESCRIPCIÓN
GET	/	Público	Health check — verifica que el servicio esté en ejecución y responde con el estado del MS
POST	/auth/login	Público	Recibe credenciales en formato application/x-www-form-urlencoded (OAuth2), valida contra BD y retorna JWT

EXPONE (RPC Server)

RPC

Otros MS llaman a estas colas

rpc_auth_queue

Acción: validate_token

Todos los MS llaman a esta cola para validar el JWT del usuario antes de procesar cualquier petición protegida.

CONSUME (RPC Client)

RPC

Este MS llama a colas externas

No aplica para este microservicio

EVENTOS Pub/Sub

EVT

Publicaciones asíncronas

auth.reset_password

Se publica cuando un usuario solicita restablecimiento de contraseña. ms-notificaciones lo recibe y envía el enlace de recuperación por correo.

CARACTERISTICA DESTACADA

Autenticación centralizada con JWT y validación distribuida vía RabbitMQ RPC

El token JWT contiene el rol del usuario en su payload. Cada microservicio valida el token en tiempo real llamando a ms-auth por RPC — sin base de datos compartida, sin acoplamiento directo.

1

El frontend envía email y contraseña al gateway (/api/auth/login) en formato form-encoded

2

ms-auth valida las credenciales contra agm_auth_db y genera un JWT firmado con HS256

3

El JWT contiene: sub (ID de usuario), rol (Administrador/Docente/Alumno) y exp (expiración)

4

El frontend adjunta el token en cada request: Authorization: Bearer <token>

5

Cada MS llama a rpc_auth_queue con el token; ms-auth responde con los datos del usuario o rechaza

6

Si el token es inválido o expirado: HTTP 401. Si el rol no tiene permiso: HTTP 403

FRAMEWORK

Django 5 · Python 3.12

PUERTO

:8001

BASE DE DATOS

agm_periodos_materias_db

DESCRIPCION DEL MICROSERVICIO

Microservicio que gestiona el catálogo académico central: los periodos lectivos y las materias que se imparten en cada uno. Es el único MS del proyecto construido con Django y Django REST Framework en lugar de FastAPI.

Al ser el catálogo de referencia, casi todos los demás microservicios (calificaciones, asistencias, docentes, reportes) lo consultan vía RabbitMQ RPC para obtener datos de materias. También es responsable de los eventos de inscripción (bienvenida) y cierre de materia.

Expone `rpc_periodos_queue` con cuatro operaciones: obtener materia por ID, obtener materia por NRC, obtener periodo activo y listar materias por docente.

MÉTODO	ruta	ROL REQUERIDO	DESCRIPCIÓN
GET	/periodos/	Administrador	Lista todos los periodos académicos registrados en el sistema
POST	/periodos/	Administrador	Crea un nuevo periodo académico con fecha de inicio y fin
GET	/periodos/{id}/	Admin / Docente	Obtiene el detalle de un periodo específico por su ID
PUT	/periodos/{id}/	Administrador	Actualiza los datos de un periodo existente
DELETE	/periodos/{id}/	Administrador	Elimina un periodo si no tiene materias activas asociadas
GET	/materias/	Admin / Docente	Lista materias, con filtros por periodo y por docente asignado
POST	/materias/	Administrador	Crea una materia y la asigna a un periodo y a un docente
GET	/materias/{id}/	Admin / Docente	Detalle completo de una materia incluyendo NRC y docente asignado
PUT	/materias/{id}/	Administrador	Actualiza los datos de una materia (nombre, NRC, docente, créditos)
DELETE	/materias/{id}/	Administrador	Elimina una materia del catálogo

EXPONE (RPC Server)

RPC

Otros MS llaman a estas colas

```
rpc_periodos_queue  
get_materia_by_id  
get_materia_by_nrc  
get_periodo_activo  
list_materias_by_docente
```

CONSUME (RPC Client)

RPC

Este MS llama a colas externas

No aplica para este microservicio

EVENTOS Pub/Sub

EVT

Publicaciones asíncronas

```
periodos.bienvenida  
Al inscribir un alumno a una materia — ms-  
notificaciones envía correo de bienvenida.  
  
periodos.materia_cerrada  
Al cerrar una materia — ms-notificaciones avisa a  
los alumnos inscritos.
```

CARACTERISTICA DESTACADA

Fuente de verdad del catálogo académico — consultada por 4 microservicios distintos

ms-periodos-materias es el origen de datos de materias y periodos para todo el sistema. Ningún otro MS almacena datos de materias en su propia BD; todos los consultan por RPC cuando los necesitan.

1

ms-calificaciones consulta `get_materia_by_id` para verificar el docente propietario (anti-IDOR)

2

ms-asistencias consulta `get_materia_by_id` para validar la existencia de la materia al abrir sesión

3

ms-reportes consulta `get_materia_by_id` para obtener nombre, NRC y docente del reporte generado

4

ms-docentes consulta `list_materias_by_docente` al importar listas de alumnos desde Excel

5

Al inscribir alumno: publica `periodos.bienvenida` para que ms-notificaciones envíe el correo

6

Al cerrar materia: publica `periodos.materia_cerrada` para notificar a todos los alumnos

FRAMEWORK

FastAPI · Python 3.12

PUERTO

:8003

BASE DE DATOS

agm_docentes_db

DESCRIPCION DEL MICROSERVICIO

Microservicio encargado del catálogo de docentes y alumnos. Permite registrar y gestionar docentes de forma individual o mediante la importación automática de la programación académica en PDF. Los alumnos pueden registrarse individualmente o cargarse de forma masiva mediante un archivo Excel con la lista de inscritos.

Expone `rpc_docentes_queue` para que otros microservicios (asistencias, calificaciones, reportes) puedan consultar qué alumnos pertenecen a una materia determinada, sin acceder directamente a su base de datos.

Es el MS que conoce la relación alumno-materia, que es esencial para validar inscripciones y calcular estadísticas.

MÉTODO	ruta	ROL REQUERIDO	DESCRIPCIÓN
GET	/docentes/	Administrador	Lista todos los docentes registrados en el sistema
POST	/docentes/	Administrador	Registra un nuevo docente con sus datos académicos
GET	/docentes/{id}/	Admin / Docente	Obtiene el perfil completo de un docente por ID
PUT	/docentes/{id}/	Administrador	Actualiza los datos de un docente existente
DELETE	/docentes/{id}/	Administrador	Elimina un docente del sistema
GET	/alumnos/	Admin / Docente	Lista alumnos con filtro opcional por materia (NRC)
POST	/alumnos/	Administrador	Registra un alumno individualmente en una materia
POST	/alumnos/importar/	Administrador	Importa lista masiva de alumnos desde archivo Excel
POST	/importar-pdf/	Administrador	Parsea PDF de programación académica y registra materias y docentes
DELETE	/alumnos/{id}/baja/	Administrador	Da de baja a un alumno y publica evento docentes.baja

EXPONE (RPC Server)

RPC

Otros MS llaman a estas colas

`rpc_docentes_queue`
`get_alumnos_by_materia`
Retorna la lista de alumnos inscritos a una materia.
Utilizado por ms-asistencias, ms-calificaciones y ms-reportes.

CONSUME (RPC Client)

RPC

Este MS llama a colas externas

`rpc_periodos_queue`
`get_materia_by_id`
Para validar que la materia existe en el catálogo
antes de asociar alumnos en la importación desde Excel.

EVENTOS Pub/Sub

EVT

Publicaciones asíncronas

`docentes.baja`
Se publica cuando se da de baja a un alumno de una materia. ms-notificaciones envía un aviso al docente responsable de la materia.

CARACTERISTICA DESTACADA

Importación automática de PDF de programación académica y PDF de alumnos

Los administradores cargan directamente los archivos oficiales de la institución. ms-docentes los parsea, valida los datos contra ms-periodos-materias y los registra automáticamente, eliminando la captura manual.

1

Admin sube el PDF de programación académica: el sistema extrae NRC, nombre, horario, aula y docente de cada materia

2

Se valida cada NRC contra `rpc_periodos_queue` para asegurar que la materia existe en el catálogo

3

Admin sube el Excel de alumnos: el sistema lee matrícula, nombre y NRC de materia inscrita por fila

4

Por cada alumno inscrito, se llama a `ms-periodos-materias` y se publica el evento `periodos.bienvenida`

5

Al dar de baja a un alumno, se publica `docentes.baja` para notificar al docente por correo

6

`rpc_docentes_queue` queda disponible inmediatamente para que otros MS consulten la relación alumno-materia

FRAMEWORK

FastAPI · Python 3.12

PUERTO

:8004

BASE DE DATOS

agm_calificaciones_db

DESCRIPCION DEL MICROSERVICIO

Microservicio encargado del registro, cálculo y consulta de calificaciones. El flujo de trabajo es el siguiente: el docente primero define las actividades evaluativas de su materia (exámenes, tareas, proyectos) con su porcentaje de ponderación. Una vez definidas, registra las calificaciones de cada alumno por actividad. El sistema calcula automáticamente el promedio ponderado.

Incorpora una protección anti-IDOR que impide que un docente acceda o modifique las calificaciones de una materia que no le fue asignada. Para esto, consulta ms-periodos-materias por RPC y compara el docente del token con el docente dueño de la materia.

Soporta importación masiva de calificaciones desde un archivo Excel con la estructura predefinida.

MÉTODO	RUTA	ROL REQUERIDO	DESCRIPCIÓN
POST	/actividades	Docente / Admin	Crea una actividad evaluativa con nombre, descripción y porcentaje de ponderación
GET	/actividades/{materia_id}	Docente / Admin	Lista todas las actividades definidas para una materia con sus ponderaciones
POST	/calificaciones	Docente	Registra la calificación de un alumno en una actividad específica
POST	/calificaciones/importar	Docente	Importa calificaciones en lote desde un archivo Excel con formato predefinido
GET	/calificaciones/{materia_id}/{alumno_id}/promedio	Docente / Alumno	Calcula el promedio ponderado del alumno validando que las ponderaciones sumen 100%
GET	/concentrado/{materia_id}	Docente / Admin	Genera el concentrado completo: todos los alumnos con sus calificaciones por actividad

EXPONE (RPC Server)

RPC

Otros MS llaman a estas colas

No aplica para este microservicio

CONSUME (RPC Client)

RPC

Este MS llama a colas externas

rpc_periodos_queue
get_materia_by_id
Para obtener el docente asignado a la materia y verificar que el solicitante es el propietario (anti-IDOR).

rpc_docentes_queue
get_alumnos_by_materia
Para obtener la lista de alumnos al generar el concentrado de calificaciones.

EVENTOS Pub/Sub

EVT

Publicaciones asíncronas

No aplica para este microservicio

CARACTERÍSTICA DESTACADA

Protección anti-IDOR + Ponderaciones que deben sumar exactamente 100%

El sistema implementa dos controles de negocio críticos: protección contra acceso no autorizado a materias ajenas (IDOR), y validación de consistencia en el esquema de ponderaciones antes de calcular el promedio final.

1

Docente solicita ver o modificar calificaciones de la materia X

2

ms-calificaciones llama a `rpc_periodos_queue: get_materia_by_id(X)`
— obtiene el docente asignado

3

Si el `docente_id` del JWT no coincide con el docente de la materia:
HTTP 403 'Acceso denegado'

4

Para calcular el promedio: el sistema verifica que la suma de ponderaciones de las actividades sea exactamente 100%

5

Si las ponderaciones no suman 100%: el endpoint retorna error explicando qué ponderaciones faltan o sobran

6

Importación Excel: valida el formato fila por fila, calcula y persiste todas las calificaciones en una transacción

FRAMEWORK

FastAPI · Python 3.12 + Redis 7

PUERTO

:8005

BASE DE DATOS

attendance_db

DESCRIPCION DEL MICROSERVICIO

Microservicio de control de asistencias mediante códigos QR con ventana de tiempo controlada. El docente abre una sesión de asistencia para su materia; la sesión queda activa durante 10 minutos en Redis. Durante ese tiempo, cada alumno genera su propio token QR desde la aplicación y lo presenta para registrar su asistencia.

El sistema implementa protección anti-replay: cada token QR puede usarse una sola vez. Una vez escaneado y registrado, el token se marca como consumido en Redis y no puede volver a utilizarse, aunque el alumno lo comparta con otro.

El sistema distingue entre Presente (llegada en los primeros 5 minutos) y Retardo (llegada entre los minutos 5 y 10). Al registrar un retardo, se publica un evento asíncrono para notificar al docente.

MÉTODO	ROUTA	ROL REQUERIDO	DESCRIPCIÓN
POST	/sesiones/iniciar	Docente	Abre una sesión de asistencia de 10 minutos para una materia; la guarda en Redis con TTL
DELETE	/sesiones/{materia_id}/cerrar	Docente	Cierra la sesión activa antes de que expire el tiempo (cierre manual)
POST	/qr/generar	Alumno	Genera un token QR único y firmado para el alumno en la sesión activa de su materia
POST	/asistencias/registrar	Alumno	Registra la asistencia presentando el token QR; valida inscripción y aplica anti-replay
GET	/asistencias/{materia_id}/hoy	Docente	Lista en tiempo real los alumnos presentes y con retardo en la sesión activa
GET	/asistencias/{materia_id}/historial	Docente / Admin	Historial paginado de todas las sesiones anteriores con sus registros de asistencia

EXPONE (RPC Server)

RPC

Otros MS llaman a estas colas

rpc_asistencias_queue
get_estadisticas_asistencia
get_asistencia_alumno
Utilizado por ms-reportes para obtener datos de sesiones y asistencias al generar reportes de una materia.

CONSUME (RPC Client)

RPC

Este MS llama a colas externas

rpc_docentes_queue
get_alumnos_by_materia
Para verificar que el alumno que presenta el QR está efectivamente inscrito en la materia antes de registrar su asistencia.

EVENTOS Pub/Sub

EVT

Publicaciones asíncronas

asistencias.retardo
Se publica en events_exchange cuando un alumno registra asistencia entre los minutos 5 y 10 de la sesión. ms-notificaciones avisa al docente.

CARACTERÍSTICA DESTACADA

Sesiones QR con ventana de tiempo y protección anti-replay mediante Redis

La combinación de Redis para sesiones efímeras y la validación de inscripción por RPC garantiza que solo los alumnos correctos, en el momento correcto, puedan registrar su asistencia, sin posibilidad de suplantación.

1

Docente inicia sesión: se guarda {materia_id, inicio} en Redis con TTL de 600 segundos (10 minutos)

2

Alumno genera QR: el sistema crea un token único firmado con alumno_id + materia_id + timestamp

3

Alumno presenta QR: ms-asistencias verifica que la sesión de esa materia sigue activa en Redis

4

Verifica inscripción llamando a rpc_docentes_queue — si el alumno no está inscrito: rechaza la asistencia

5

Verifica que el token no fue usado antes (anti-replay); si ya fue usado: rechaza con error específico

6

Minutos 0–5: estado 'Presente'. Minutos 5–10: estado 'Retardo' + publica asistencias.retardo

FRAMEWORK

Node.js 20 · Express

PUERTO

Interno

BASE DE DATOS

agm_notificaciones_db

DESCRIPCION DEL MICROSERVICIO

Microservicio de notificaciones por correo electrónico. Es el único MS del proyecto construido con Node.js. A diferencia del resto, no expone ningún puerto HTTP — opera exclusivamente como consumidor de eventos del bus RabbitMQ.

Cuando algún otro MS publica un evento en `events_exchange`, ms-notificaciones lo recibe, identifica el tipo de evento, formatea el correo correspondiente y lo envía a través de SMTP4dev (entorno de desarrollo).

Su diseño es intencionalmente desacoplado: ningún otro MS lo llama directamente ni depende de su disponibilidad. Si falla, el sistema completo sigue operando con normalidad. Solo se pierden los correos mientras el servicio esté caído.

Consume seis tipos de eventos distintos, cada uno con su propio handler y plantilla de correo.

MÉTODO	RUTA	ROL REQUERIDO	DESCRIPCIÓN
—	Sin endpoints HTTP	—	Este microservicio no expone ningún puerto ni ruta REST. Opera exclusivamente como consumidor de eventos de RabbitMQ. No puede ser llamado directamente.

EXPONE (RPC Server)

RPC

Otros MS llaman a estas colas

No aplica para este microservicio

CONSUME (RPC Client)

RPC

Este MS llama a colas externas

No aplica para este microservicio

EVENTOS Pub/Sub

EVT

Publicaciones asíncronas

Consume todos los eventos del exchange 'events_exchange':

auth.reset_password — Envía enlace de recuperación de contraseña
periodos.bienvenida — Correo de bienvenida al alumno inscrito
periodos.materia_cerrada — Aviso de cierre de materia a alumnos
docentes.baja — Notificación al docente sobre baja de alumno
asistencias.retardo — Aviso de retardo al docente
reportes.generado — Aviso de reporte disponible para descarga

CARACTERISTICA DESTACADA

Consumidor de eventos puro — Totalmente desacoplado del resto del sistema

ms-notificaciones nunca es invocado directamente. Escucha el bus de eventos y reacciona a 6 tipos de eventos distintos, cada uno con su plantilla de correo. Su caída no afecta ningún flujo principal del sistema.

1

Al arrancar, conecta a RabbitMQ y se suscribe al exchange 'events_exchange' con routing key '#' (todos los eventos)

2

Recibe 'auth.reset_password': genera enlace de recuperación y envía correo al usuario

3

Recibe 'periodos.bienvenida': formatea correo de bienvenida con datos de la materia y lo envía al alumno

4

Recibe 'asistencias.retardo': notifica al docente que un alumno llegó tarde, con el nombre y hora del registro

5

Recibe 'reportes.generado': avisa al docente que su reporte de calificaciones o asistencias está disponible

6

Cada mensaje es confirmado (ACK) solo si el correo se envió con éxito; si falla, se encola nuevamente (NACK)

FRAMEWORK

FastAPI · Python 3.12

PUERTO

:8007

BASE DE DATOS

reports_db

DESCRIPCION DEL MICROSERVICIO

Microservicio de generación de reportes en formato PDF y Excel. Para generar un reporte, obtiene los datos necesarios consultando tres microservicios distintos por RabbitMQ RPC: ms-periodos-materias (datos de la materia), ms-docentes (lista de alumnos) y ms-asistencias (registros de sesiones).

Si alguno de los microservicios consultados no responde, el endpoint retorna HTTP 503 con un mensaje descriptivo — nunca genera reportes con datos ficticios.

Además, almacena en su base de datos las estadísticas históricas de materias y alumnos.

MÉTODO	ROUTA	ROL REQUERIDO	DESCRIPCIÓN
GET	/reportes/calificaciones/{materia_id}	Admin / Docente	Genera y descarga reporte de calificaciones consultando datos reales por RPC (PDF o Excel)
POST	/reportes/calificaciones/{materia_id}	Admin / Docente	Genera reporte de calificaciones con los datos enviados directamente en el cuerpo de la petición
GET	/reportes/asistencias/{materia_id}	Admin / Docente	Genera y descarga reporte de asistencias consultando ms-asistencias por RPC (PDF o Excel)
POST	/reportes/asistencias/{materia_id}	Admin / Docente	Genera reporte de asistencias con los datos enviados en el cuerpo de la petición
GET	/estadisticas/docente/{docente_id}	Admin / Docente	Retorna historial paginado de estadísticas del docente agrupadas por periodo y materia
POST	/estadisticas/registrar	Administrador	Registra estadísticas de una materia al cerrarla: alumnos, promedio, porcentaje de aprobación
GET	/estadisticas/alumno/{alumno_id}	Admin / Docente / Alumno	Retorna estadísticas paginadas del alumno por materia: asistencias, calificación y periodo
POST	/estadisticas/alumno/registrar	Administrador	Registra las estadísticas finales de un alumno al cierre de materia
GET	/reportes/historial	Administrador	Lista paginada de todos los reportes generados con fecha, tipo, formato y docente solicitante

EXPONE (RPC Server)

RPC

Otros MS llaman a estas colas

rpc_reportes_queue
generate_calificaciones_report
generate_asistencias_report
get_historial_docente
Disponibles para que otros MS puedan solicitar reportes programáticamente.

CONSUME (RPC Client)

RPC

Este MS llama a colas externas

rpc_periodos_queue
get_materia_by_id — nombre, NRC, docente

rpc_docentes_queue
get_alumnos_by_materia — lista de alumnos

rpc_asistencias_queue
get_estadisticas_asistencia
get_asistencia_alumno

EVENTOS Pub/Sub

EVT

Publicaciones asíncronas

reportes.generado
Fire-and-forget. Se publica en events_exchange al terminar de generar cualquier reporte (GET o POST).

CARACTERISTICA DESTACADA

Generación de reportes con datos reales de 3 MS

Al generar un reporte, ms-reportes orquesta llamadas RPC a tres servicios, construye el archivo PDF o Excel y lo retorna como descarga directa. Paralelamente, publica el evento de disponibilidad sin bloquear la respuesta al usuario.

1

Frontend solicita GET /reportes/calificaciones/42?formato=pdf con JWT de docente en el header

2

ms-reportes llama a rpc_periodos_queue para obtener nombre y NRC de la materia — si no responde: HTTP 503

3

Llama a rpc_docentes_queue para obtener la lista de alumnos inscritos en la materia

4

Llama a rpc_asistencias_queue para obtener estadísticas de asistencia por alumno (solo si es reporte de asistencias)

5

Genera el archivo PDF o Excel con los datos reales y retorna la descarga directamente al frontend

6

Publica reportes.generado en events_exchange (fire-and-forget)

Conclusiones

01	ARQUITECTURA	Siete microservicios independientes, cada uno con base de datos, Dockerfile y puerto propios.
02	COMUNICACIÓN	Estrategia híbrida: RPC síncrono para operaciones que requieren respuesta inmediata, Pub/Sub asíncrono para eventos entre servicios.
03	DESACOPLAMIENTO	El sistema continúa operando aunque un servicio no crítico como ms-notificaciones falle — desacoplamiento real, no solo estructural.
04	SEGURIDAD	JWT centralizado con validación por RPC, roles estrictos, protección anti-IDOR en calificaciones y anti-replay en asistencias QR.
05	FUNCIONALIDAD	Reportes PDF/Excel con datos reales, control de asistencias con QR, importación de listas desde PDF y Excel, notificaciones por correo.
06	DESPLIEGUE	Backend desplegado en Railway, frontend en Vercel. Sistema completamente productivo y accesible desde internet.